Design and Implementation of an Intelligent Number-Guessing Agent Using Binary Search Logic

Aim

To design and implement a goal-based intelligent agent in Python that guesses a secret number chosen by the user or environment within a defined range [1, N] by utilizing binary search strategy based on qualitative feedback (Too High / Too Low).

Practical Prerequisites & Requirements

Language: Python 3.x
Operating System: Any OS (Linux, Windows, macOS)
IDE/Editor: VS Code, PyCharm, or Jupyter Notebook
Hardware: Minimum 2 GB RAM with any modern CPU

Theory

1. Intelligent Agent Concept
An agent is anything that perceives its environment through sensors and acts upon that environment through actuators. The Number-Guessing Agent is a Goal-Based Agent that maintains an internal state representing the plausible range of values and updates its actions to reach the target goal state (the correct number).

2. PEAS Framework Description
Performance Measure: Minimal number of guesses required to find the target number; speed of reaching the goal.
Environment: A discrete, deterministic, static number line between 1 and N.
Actuators: Displaying the guessed number to the user.
Sensors: Receiving directional feedback (Higher, Lower, or Correct).

3. Binary Search Strategy
Rather than searching linearly (which takes O(N) time), the agent splits the search space in half at each iteration:

Guess = floor((Low + High) / 2)

If feedback is Too High: the target must be smaller, so update High = Guess - 1.
If feedback is Too Low: the target must be larger, so update Low = Guess + 1.
If feedback is Correct: the goal state is reached.

This search reduces the candidate search space by 50% in every step, guaranteeing logarithmic performance.

Procedure

1. **Initialize Range:** Define the search boundaries Low = 1 and High = N (e.g., N = 100).
2. **Compute Midpoint:** Calculate the agent's current guess as floor((Low + High) / 2).
3. **Receive Percept (Feedback):** Prompt the user to provide feedback on whether the guess is too high,

too low, or correct.

4. **Update Search Bounds:**

• If Too High, set High = Guess - 1.

• If Too Low, set Low = Guess + 1.

• If Correct, terminate the search loop and display success metrics.

5. **Validation Check:** If Low > High, detect inconsistent user feedback and terminate with an error prompt.

Python Code

```
class NumberGuessingAgent:
    def __init__(self, low=1, high=100):
        self.low = low
        self.high = high
        self.attempts = 0

    def make_guess(self):
        """Calculates the optimal guess using binary search strategy."""
        return (self.low + self.high) // 2

    def update_bounds(self, feedback, current_guess):
        """Updates internal search boundaries based on environmental feedback."""
        self.attempts += 1
        if feedback == "H":
            self.high = current_guess - 1
        elif feedback == "L":
            self.low = current_guess + 1

    def run(self):
        print("=" * 50)
        print("    INTELLIGENT NUMBER-GUESSING AGENT (A.I.)    ")
        print("=" * 50)
        print(f"Think of a number between {self.low} and {self.high}.\n")
        print("Provide Feedback Rules:")
        print("  Enter 'H' if the agent's guess is Too HIGH")
        print("  Enter 'L' if the agent's guess is Too LOW")
        print("  Enter 'C' if the agent's guess is CORRECT\n")

        while self.low <= self.high:
            guess = self.make_guess()
            feedback = input(
                f"Agent Guess #{self.attempts + 1}: Is it {guess}? [H/L/C]: "
            ).strip().upper()

            if feedback == 'C':
                self.attempts += 1
                print(f"\n[Goal Reached] Agent found your number "
                      f"'{guess}' in {self.attempts} attempts!")
                return
            elif feedback in ['H', 'L']:
                self.update_bounds(feedback, guess)
            else:
                print("Invalid input! Please enter 'H', 'L', or 'C'.")

        print("\n[Error] Inconsistent feedback detected! The search space was exhausted.")


if __name__ == "__main__":
    agent = NumberGuessingAgent(low=1, high=100)
    agent.run()
```

Sample Output

```
==================================================
    INTELLIGENT NUMBER-GUESSING AGENT (A.I.)
==================================================
Think of a number between 1 and 100.

Provide Feedback Rules:
  Enter 'H' if the agent's guess is Too HIGH
  Enter 'L' if the agent's guess is Too LOW
  Enter 'C' if the agent's guess is CORRECT
Agent Guess #1: Is it 50? [H/L/C]: H
```

```
Agent Guess #2: Is it 25? [H/L/C]: L
Agent Guess #3: Is it 37? [H/L/C]: L
Agent Guess #4: Is it 43? [H/L/C]: H
Agent Guess #5: Is it 40? [H/L/C]: C

[Goal Reached] Agent found your number '40' in 5 attempts!
```

Analysis & Evaluation

Time Complexity:
Worst-Case: O(log■ N). For N = 100, the maximum number of attempts is ceil(log■(100)) = 7 attempts.
Best-Case: O(1) if the midpoint on the first guess happens to be the secret number.

Space Complexity: O(1) auxiliary space as the agent only maintains two integer scalar variables (low and high).

Efficiency Comparison:
Random Guessing: Unpredictable performance; no deterministic performance guarantee.
Linear Search: Requires up to N attempts, O(N).
Binary Search Agent: Guarantees logarithmic efficiency, O(log N), by eliminating half of the remaining search space with each guess.

Conclusion

The intelligent number-guessing agent was successfully implemented in Python. By applying a binary search pruning strategy based on environmental feedback, the agent consistently converges to the target goal state in worst-case logarithmic time complexity O(log■ N).

Viva Questions & Answers

Q1: What category of AI agent does this implementation represent?
Answer: It is a Goal-Based Agent. It keeps track of environment state boundaries and makes deliberate choices toward the goal.

Q2: What is the PEAS structure for this system?
Answer: P: Minimizing guesses; E: Range [1,N]; A: Printed prompts; S: User feedback H, L, C.

Q3: How many maximum attempts for range 1 to 1000?
Answer: ceil(log■(1000)) = 10 attempts.

Q4: What happens with conflicting feedback?
Answer: The boundaries cross such that Low > High, and the agent detects the inconsistency.

Q5: Why is binary search preferred over random guessing?
Answer: Binary search deterministically finds the goal in O(log■ N) steps by eliminating half the search space each time.